

BHTTP/1 — HTTP semantics in binary frames

Protocol specification, version 1 · reference implementations: `bserve` (server) and `bcurl` (client) · 2026-10-08

BHTTP/1 carries HTTP-style requests and responses (methods, paths, status codes, header fields) over a single TCP connection. Instead of text lines it uses length-prefixed binary *frames* and a small table of numbered header names. The key words **MUST**, **MUST NOT**, **SHOULD** and **MAY** are used as in RFC 2119. All multi-octet integers are unsigned and big-endian (network byte order).

1 Connection model

The client opens one TCP connection to the server (there is no registered port; **9000** is the default). It then sends any number of requests on that connection, one after another. The server **MUST** keep the connection open after each response. Only the client closes it, by closing TCP. The server **MAY** close a connection that has been idle (between frames) for 60 s or more. Requests **MAY** be pipelined, meaning a client can send several before reading any response. The server **MUST** answer them in the order received. A client **MUST NOT** start a new request until the previous request is complete, which means a frame carrying END has been sent for it.

2 Frame header

Every octet on the connection belongs to a frame. A frame is an 8-octet header followed by *Length* octets of payload:

octet	0	1	2	3	
	+-----+-----+-----+-----+				
0		Length (16)		Type Flags	
	+-----+-----+-----+-----+				
4		Request ID (32)			
	+-----+-----+-----+-----+				
8		Payload (Length octets) ...			

Field	Meaning
Length	Payload size in octets, 0–65 535. It does not count the 8-octet header.
Type	Frame type (§3). It decides how the payload is read.
Flags	Bits whose meaning depends on the type. Bit 0x01 is END on both DATA and HEADERS. Senders MUST set unassigned bits to 0, and receivers MUST ignore them.
Request ID	Identifies which request this frame belongs to. The client picks a non-zero value for each request: 1 for the first, then strictly increasing. Every frame of the response carries the same value. ID 0 is reserved for frames that concern the whole connection; v1 defines none.

A receiver **MUST** always read exactly *Length* payload octets, whatever the type. That rule is why framing can never lose its place. Whatever a payload contains, the next frame starts exactly *Length* octets after it.

3 Frame types

Type	Name	Payload
0x00	DATA	Message body octets, sent as-is. A body may span many DATA frames. The last one carries END, and it may be empty.
0x01	HEADERS	One complete header block (§4). END means no body follows.
0x02–0xFF	<i>unassigned</i>	Reserved for future versions.

Extensibility rule. A receiver that meets a frame type it does not know **MUST** skip it cleanly. It reads and discards the *Length* payload octets and continues with the next frame. This holds whatever the Request ID or Flags, and even between a request's HEADERS and DATA frames. It **MUST NOT** reply, report an error or close the connection. This is how version 2 can add frame types that a v1 peer safely ignores.

4 Header block

A header block is a sequence of fields, with no count and no terminator. The HEADERS frame's *Length* marks where the block ends. Each field is:

```
indexed (idx = 0x01..0x0A):
  | idx (1) | vlen (2) | value (vlen) |
literal:
  | 0x00 (1) | nlen (1) | name (nlen) |
  | vlen (2) | value (vlen) |
```

Indexes 0x01–0x0A name the ten fields that `bserve` and `bcurl` actually send. The name is then implied by its number, and only the value is length-prefixed:

#	Name	#	Name
1	:method	6	accept
2	:path	7	content-type
3	:status	8	content-length
4	host	9	server
5	user-agent	10	date

Any other name uses the literal form: 0x00, a 1-octet name length (1–255), then the name. Names **MUST** be lower-case printable ASCII without spaces. Values are 0–65 535 arbitrary octets; header values in this version are ASCII text. A sender **SHOULD** use the index whenever one exists. A receiver **MUST** also accept the literal spelling of a table name. Receivers **MUST** ignore fields they do not understand.

A block is **malformed** in any of these cases:

- A length runs past the end of the block.
- A prefix octet is in the unassigned range 0x0B–0xFF.
- A literal name is empty or contains a forbidden octet.

A whole block **MUST** fit in one HEADERS frame, so its limit is 65 535 octets.

5 Messages

Request. A request is one HEADERS frame. It **MUST** contain `:method` and `:path` exactly once each, and `:path` **MUST** start with `/`. It **SHOULD** contain `host`. If END is clear, a body follows in DATA frames, the last of which carries END. Methods: `GET` and `HEAD`. The server reads and discards any request body, because v1 defines no method that uses one.

Response. A response is one HEADERS frame, with `:status` as three ASCII digits. DATA frames follow until one carries END. If there is no body (HEAD, an empty file), the HEADERS frame itself carries END. The server **SHOULD** send `content-length`, but END is what delimits the body. A response **MUST** carry the Request ID of the request it answers.

6 Server behaviour

Usage: `bserve ROOT PORT`. To resolve `:path` to a file, the server:

1. Removes everything from the first `?` or `#` onwards.
2. Decodes `%XX` escapes. A bad escape, or one that decodes to NUL, is malformed.
3. Appends the decoded path to ROOT. A trailing `/` means `index.html`.

- 4. Resolves symlinks and `...`. The result **MUST** lie inside ROOT, or the server returns 404. A directory maps to its `index.html`.
- 5. Serves the result if it is a regular, readable file, and returns 404 otherwise.

Status When

200	File found. The body is its octets, with <code>content-type</code> chosen from the file extension.
400	Malformed request, which is any of: • a malformed header block • <code>:method</code> or <code>:path</code> missing or repeated • a path that does not start with <code>/</code> • a bad escape • Request ID 0 • DATA that belongs to no open request • HEADERS sent while a request body is still open
404	No such file, not a regular file, or outside ROOT.
405	A method other than GET or HEAD.

Error responses carry a short `text/plain` body such as `404 Not Found`. A **400 does not close the connection**. Since every frame's length was honoured (§2), a malformed request costs only its own frames, and the next request is served normally. The only connection-level failure is TCP closing or timing out in the middle of a frame. In that case no frame boundary can be trusted, so the server closes the connection.

7 Client behaviour

Usage: `bcurl [-v] [-I] [-g] URL [URL | /path ...]`. Here URL is `[bhttp://]host[:port] [/path]`.

- The client opens **exactly one** connection and sends every request on it, in order, with IDs 1, 2, 3 and so on.
- Every URL must name the same host and port. This is checked before the client dials, so a mismatched URL exits with status 2 without a connection attempt.
- Response bodies go to stdout.
- `-v` writes a summary and full hexdump of every frame sent (`>`) and received (`<`) to stderr, including frames that were skipped.
- `-I` sends HEAD and prints the response headers.
- Exit status: 0 if every response is 1xx–3xx, 4 if any is 4xx, 5 if any is 5xx. Status 1 means a network or protocol error. Status 2 means a usage error.

The client treats each of these as a protocol error: a response frame with a Request ID it never sent, DATA before HEADERS, a missing `:status`, or a connection that closes before END.

8 Why these widths? (HTTP/2 chose 24/8/8/31)

HTTP/2's 9-octet header has these fields:

- **24-bit length**. Peers start at a 16 KiB frame limit and can negotiate up to 16 MiB with SETTINGS.
- **8-bit type and 8-bit flags**. These leave room for extensions.
- **A reserved bit plus a 31-bit stream ID**. Many concurrent streams are interleaved on one connection, with odd IDs for the client and even IDs for the server. Keeping the top bit clear lets languages without unsigned 32-bit integers, such as Java, store the ID safely.

BHTTP/1 has no multiplexing and no negotiation, so each field is sized for the job it actually does:

Field	Choice and defence
Length 16	With nothing negotiated, the field's width is the frame limit. 64 KiB per frame keeps bulk-transfer overhead to $8/65\,543 \approx 0.012\%$. It also means every receiver can hold any frame, including one it is skipping, in a fixed 64 KiB buffer. A 24-bit field without SETTINGS would only make every receiver accept 16 MiB frames from any stranger.
Type 8	Two types are used, and 254 are left for v2. That is plenty, and the skip rule makes each new type safe to deploy.
Flags 8	One bit is used today, and the other seven are reserved for each type's future needs, such as a "compressed" bit on DATA.
Request ID 32	Without multiplexing, the ID is still useful in three ways: • A client can detect a desynchronised or misrouted response. • ID 0 leaves a place for connection-level frames. • v2 can add multiplexing without changing the header layout. There is no reserved bit, because C, Go, Python and Rust all have unsigned 32-bit integers.
Total 8	Two 32-bit words. The header can be parsed with two aligned loads, and in a 16-octet hexdump row the header is always the first 8 octets of the row.

Header names. Numbering the ten names we send is the first HPACK mechanism, the static table. Length-prefixing the rest is the second, string literals. Together they shrink the example request from 84 octets of HTTP/1.1 text to 61, and the response header from 134 to 89. Values are never compressed: there is no Huffman coding and no dynamic table, so there is no compression state shared between the two ends to get out of step.

9 Room for version 2

- **Adding frame types.** Because unknown types are skipped (§3), v2 can introduce new ones, for example a SETTINGS frame on ID 0. A v1 peer ignores them, so a v2 client can send them speculatively. If the server's reply never uses them, the server speaks v1.
- **Adding flags.** Unknown flags are ignored, so new flags work the same way.
- **Adding static-table indexes.** This is *not* safe without negotiation. An unknown index makes a block malformed, because the receiver could not know the field's name. New indexes must therefore wait until a v2-only frame has confirmed that the peer understands them.

10 Worked example

This is the request frame for `GET /index.html`, in hex. The first line is the frame header, and each line after it is one header field:

00 35 01 01 00 00 00 01	frame header
01 00 03 "GET"	:method
02 00 0b "/index.html"	:path
04 00 0e "localhost:9000"	host
05 00 07 "bcurl/1"	user-agent
06 00 03 "*/"	accept

The header gives Length `0x35 = 53`, type HEADERS, flags END and ID 1 (`:method`) has value length 3, field 2 (`:path`) has length 11, and so on. The 8-octet header plus 53 payload octets makes 61 octets on the wire. `docs/HEXDUMP` annotates every octet of a full exchange.

Conformance: `make test` runs 42 checks from an independent Python implementation of this document. It drives bserve with hand-built valid, malformed and unknown frames, and drives bcurl against a deliberately awkward Python server.